

Predicting fraudulent credit-card transactions

HarvardX PH125.9x choose-your-own capstone

Dean Grover

2026-05-10

Contents

1	Executive summary	2
2	Data and Prediction Target	2
3	Evaluation Metrics	3
4	Exploratory Analysis	3
4.1	Amount and Distance	3
4.2	Category Patterns	4
4.3	Time of Day	4
5	Feature Engineering	4
6	Modeling Strategy	4
6.1	What the Random Forest Used	6
7	Validation Results	7
8	Final Held-Out Test	8
9	Inspecting Individual Predictions	10
9.1	Correct Alerts	10
9.2	False Alerts and Missed Fraud	11
10	Discussion	11
11	Conclusion	12

12 Reproducibility	13
12.1 How to reproduce these results	13
12.2 Determinism	13
12.3 Run footprint	14
13 References	15

1 Executive summary

This project builds a fraud-screening model for credit-card transactions using the `pointe77/credit-card-transaction` dataset from Hugging Face. The question is practical: **if a stream of card transactions arrives tomorrow, can a model separate the few suspicious transactions from the many normal ones well enough to create a useful review queue?**

The headline result is concentration. The final alert queue is about 180.3 times richer in fraud than the average test transaction. The base fraud rate in the test sample is only 0.00459, while the selected alert rule has precision 0.827 — a transaction the model flags is far more likely to be fraud than a random one.

The final model is `rf_base`, a random forest fit on transaction amount, time-of-transaction, customer geography and demographics, and category. As a *rule*, it flags 1228 of 300,000 test transactions; 1016 of those are fraud and 212 are not, while 361 fraud cases slip through (recall 0.738, F1 0.780).

As a *score*, the model ranks fraud well. ROC-AUC — the chance the model gives a random fraud transaction a higher score than a random non-fraud transaction — is 0.994. PR-AUC — average precision across all threshold choices, which equals the fraud base rate (about 0.00459) for a ranker that scores at random — is 0.823, roughly 179× better than chance. Picking how aggressive the alert rule should be is an operational decision rather than a modeling one; §7 reports the same model under several alternative rules. Full definitions in §3.

2 Data and Prediction Target

Each row is a transaction. The target is `is_fraud`, where 1 means the transaction is labeled fraudulent and 0 means it is not. The dataset already provides separate train and test files, so the project keeps that split: all exploration, feature construction, model selection, and threshold selection are driven by the training data. The test set is reserved for the final estimate of performance.

The report runs against a sample of each provided file so iteration stays fast; raising the `CC_TRAIN_N` and `CC_TEST_N` environment variables (or unsetting them) re-runs the same workflow against the full files.

Table 1: Prepared train/test data

Split	Rows	Fraud	Not fraud	Fraud rate
Train	9e+05	5090	894910	0.00566
Test	3e+05	1377	298623	0.00459

The class imbalance is the central modeling issue: only 1377 of 300000 test transactions are fraud. The next section explains how the report handles that imbalance through metric choice rather than through resampling.

3 Evaluation Metrics

The report therefore uses three groups of metrics. A quick plain-English glossary:

- **Risk score:** the model’s estimate that a transaction looks suspicious. Higher score means “review this sooner.”
- **Threshold:** the cutoff used to turn scores into alerts. If the threshold is 0.80, a transaction scoring 0.80 or higher is flagged.
- **Precision:** of the transactions we flag, how many are actually fraud? This measures alert quality.
- **Recall:** of all real fraud cases, how many did we catch? This measures how much fraud the system misses.
- **F1:** a single score that balances precision and recall. It is useful when we want both a clean queue and good fraud coverage.
- **ROC-AUC:** a ranking metric. If we randomly pick one fraud transaction and one non-fraud transaction, ROC-AUC is roughly the chance the model gives the fraud transaction a higher score.
- **PR-AUC:** another ranking metric, but more useful for rare fraud. It summarizes precision and recall across many thresholds. A random ranking has PR-AUC close to the fraud base rate, which is only 0.00459 in the test data.

The operational decision is the threshold. A bank could review the top 0.5% of risk scores, the top 1%, or use a validation-selected threshold that balances precision and recall. This project reports several such rules instead of pretending there is only one definition of “accuracy.”

Table 2: Why accuracy alone is not enough for this problem

Rule	Flagged	Fraud caught	Fraud missed	Accuracy	Precision	Recall
Predict every transaction as legitimate	0	0	1377	0.9954	NA	0.0000
Final selected fraud-alert rule	1228	1016	361	0.9981	0.8274	0.7378

4 Exploratory Analysis

4.1 Amount and Distance

I first checked whether the labels looked learnable at all. If fraud and non-fraud had the same amount distribution, same categories, and same timing, then a strong classifier would be unlikely. The training data does show differences. Fraud rows have a much larger average transaction amount than legitimate rows, and the customer-to-merchant distance differs as well — though both are weak signals on their own; the model needs to combine them with category and timing to be useful.

Table 3: Training-set transaction summaries by label

is_fraud	n	median_amt	mean_amt	p90_amt	median_distance	mean_distance
Not fraud	894910	47.18	67.65	134.11	78.24	76.13
Fraud	5090	387.45	529.87	1027.30	77.97	76.48

The model uses `amt_log` rather than raw `amt` because the dollar distribution has a long right tail; the log transform gives a smoother numeric input. The amount-only baseline in §6 also follows from this picture — if the dollar distributions for fraud and non-fraud are this distinct, a rule that just ranks by amount should already do better than chance.

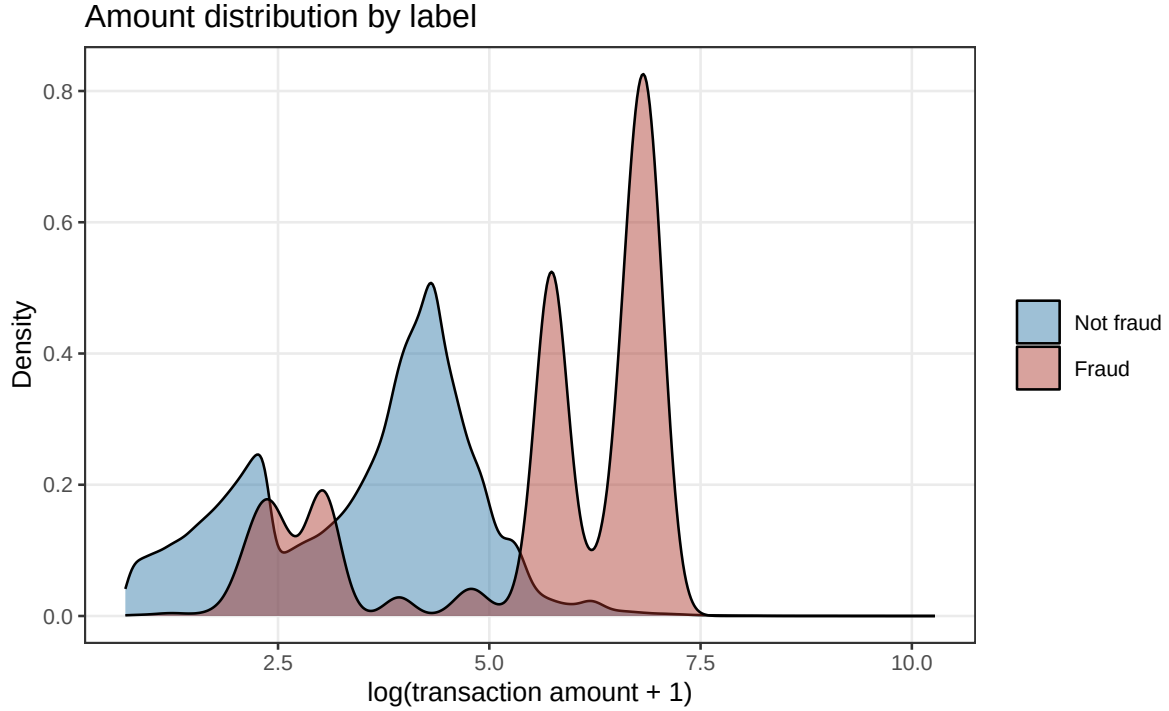


Figure 1: Transaction amount is heavily skewed, so the model uses $\log(\text{amount} + 1)$.

4.2 Category Patterns

Transaction category is a strong signal. Some categories have much higher fraud rates than others, which suggests that a useful model should not treat all purchases the same. A high-dollar grocery purchase and a high-dollar online shopping transaction may not carry the same risk.

Table 4: Highest-fraud-rate categories in the training set

category	rate	n	fraud
shopping_net	0.0173	67662	1173
misc_net	0.0139	43879	609
grocery_pos	0.0137	85498	1174
shopping_pos	0.0070	80920	569
gas_transport	0.0047	91343	430
travel	0.0030	28208	86
misc_pos	0.0030	55374	167
grocery_net	0.0030	31489	93

4.3 Time of Day

Fraud risk also varies by hour. The same amount in the same category can have a different risk profile depending on when it occurs, so hour is a useful context variable for the model alongside amount and category.

5 Feature Engineering

The preparation script creates features that match the exploratory findings and remain available for new transactions:

- **amt_log**: log-transformed transaction amount.
- **hour, day, month, weekday**: time features from the transaction timestamp.
- **age_years**: customer age at transaction time.

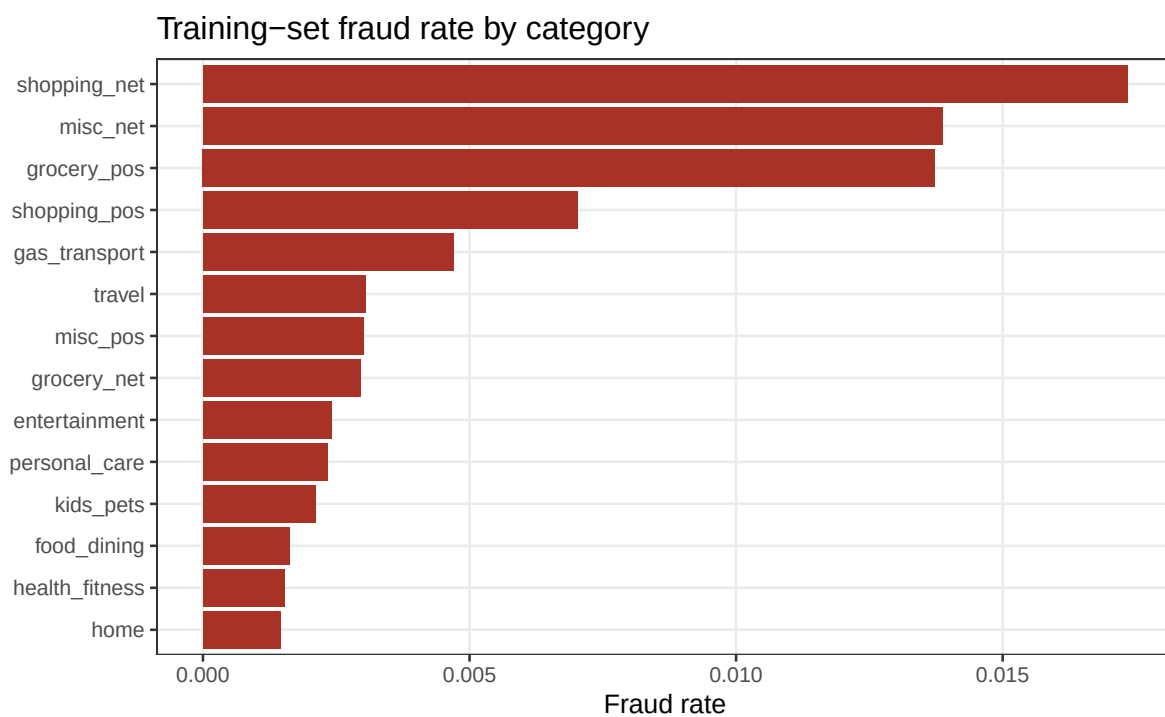


Figure 2: Fraud rate by transaction category in the training set.

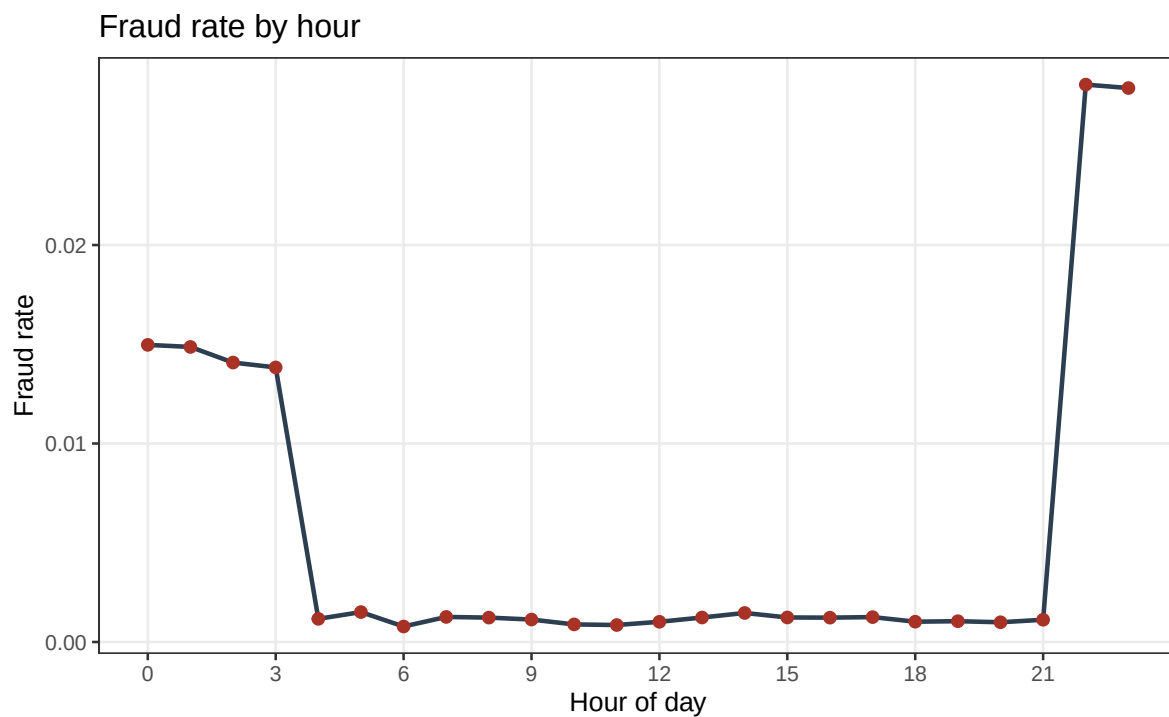


Figure 3: Hourly fraud rate in the training set.

1. Use the provided training file for learning and validation.
2. Split the training file into an internal fit set and validation set.
3. Fit a sequence of models of increasing complexity.
4. Select the best model and threshold using validation scores and validation labels.
5. Refit the selected model on the full training sample.
6. Score the held-out test sample once, using the threshold chosen during validation.

The candidates were:

- **amount_rule**: ranks transactions only by transaction amount. It asks, “How far can we get if we only assume bigger transactions are riskier?”
- **glm_base**: logistic regression using structured transaction, time, demographic, geography, and category features. A transparent linear baseline that weights each feature additively.
- **glm_encoded**: logistic regression plus train-only identity and entity risk encodings. This tests whether past risk patterns by merchant, card, job, category, or state add value.
- **rf_base**: random forest using the structured features. A random forest is many decision trees voting together; it can learn combinations such as “high amount at a certain hour in a certain category.”
- **rf_encoded**: random forest plus the identity/entity encodings.

This sequence is important. It shows whether performance comes from one obvious signal, from interpretable linear effects, from nonlinear interactions, or from richer identity history.

Table 5: Best validation result for each model family

model	rule	roc_auc	pr_auc	flagged	precision	recall	f1
rf_base	recall80	0.9961	0.8884	1125	0.9084	0.8028	0.8524
rf_encoded	best_f1	0.9955	0.8742	1114	0.8671	0.7588	0.8094
glm_encoded	best_f1	0.8863	0.4407	871	0.5901	0.4038	0.4795
glm_base	best_f1	0.8375	0.3429	830	0.5771	0.3763	0.4555
amount_rule	best_f1	0.8323	0.1779	1543	0.3603	0.4368	0.3949

Reading the table top-to-bottom: the **amount-only rule** reaches PR-AUC 0.178 against a training base rate of 0.00566 — a real signal but not a usable alert. **Logistic regression** on the structured features lifts validation F1 from 0.395 (amount rule) to 0.456, mainly by adding category, time, and geography. **Adding identity/entity encodings** to logistic regression lifts F1 to 0.479 — a modest gain.

The biggest jump is non-linearity: the **plain random forest** lands at F1 0.852, roughly 78% above the best linear variant. Notably, the **encoded random forest** (F1 0.809) does not improve on the plain version (F1 0.852) — the tree structure already captures most of what the encodings expose, and the final selected model is the simpler **rf_base**, chosen on validation before the test set was touched.

6.1 What the Random Forest Used

The selected model was not chosen only because it had the best validation score. Its strongest variables also make sense given the exploratory analysis. The random forest importance measure below is based on mean decrease in Gini impurity: higher values indicate variables that were more useful for splitting transactions into lower-risk and higher-risk groups.

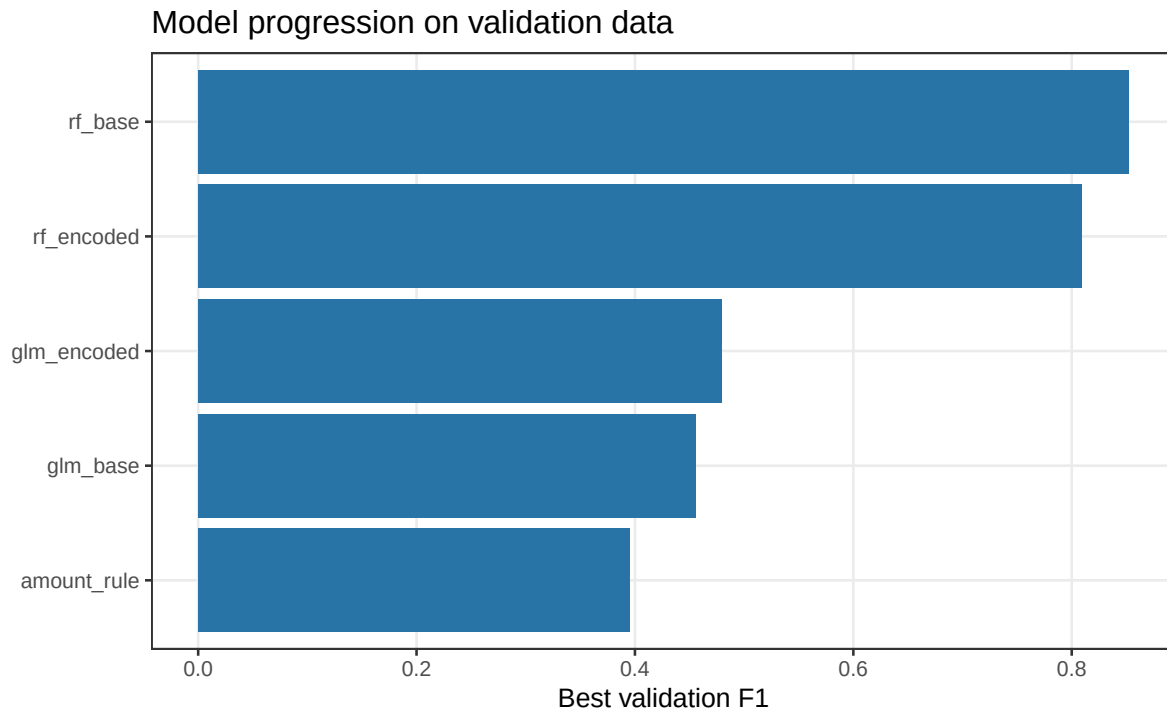


Figure 4: Validation F1 improves as the model moves from a simple amount rule to random forests.

Table 6: Most important variables in the selected final model

feature	importance
amt_log	4165.01
hour	1150.05
category	1067.14
state	390.12
age_years	229.13
city_pop_log	134.04
month	111.61
distance_km	109.69
day	102.00
weekday	100.20
gender	40.95

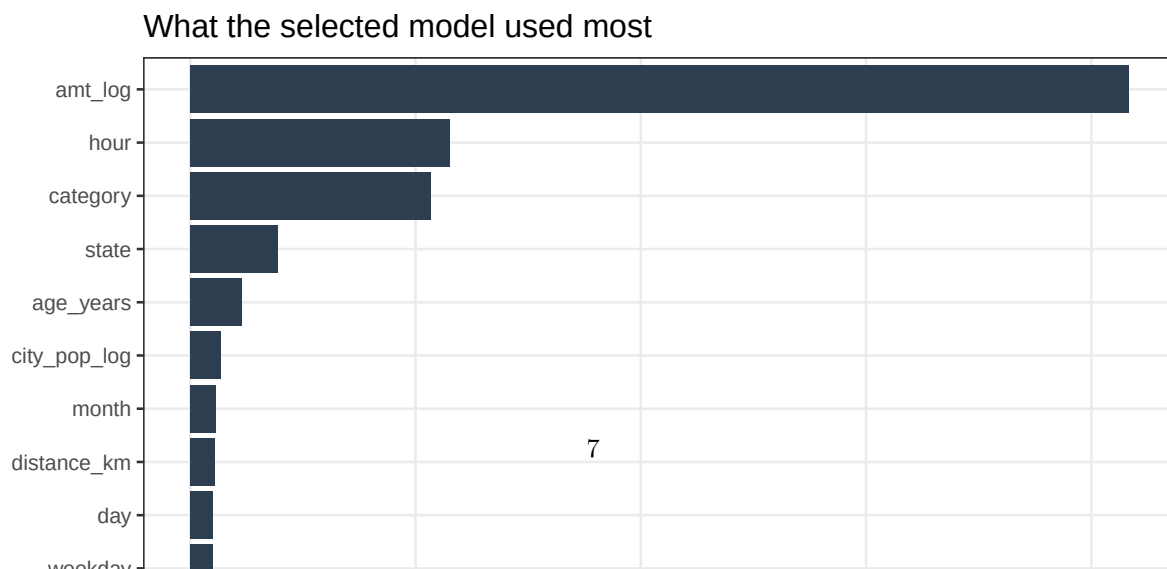


Table 7: Top validation results by F1

model	rule	roc_auc	pr_auc	flagged	tp	fp	fn	accuracy	precision	recall	f1
rf_base	recall80	0.9961	0.8884	1125	1022	103	251	0.9984	0.9084	0.8028	0.8524
rf_base	best_f1	0.9961	0.8884	1147	1031	116	242	0.9984	0.8989	0.8099	0.8521
rf_base	top_0.5pct	0.9961	0.8884	1133	1025	108	248	0.9984	0.9047	0.8052	0.8520
rf_encoded	best_f1	0.9955	0.8742	1114	966	148	307	0.9980	0.8671	0.7588	0.8094
rf_encoded	top_0.5pct	0.9955	0.8742	1129	970	159	303	0.9979	0.8592	0.7620	0.8077
rf_encoded	recall80	0.9955	0.8742	1299	1022	277	251	0.9977	0.7868	0.8028	0.7947
rf_base	top_1pct	0.9961	0.8884	2257	1145	1112	128	0.9945	0.5073	0.8995	0.6487
rf_encoded	top_1pct	0.9955	0.8742	2264	1145	1119	128	0.9945	0.5057	0.8995	0.6474
glm_encoded	best_f1	0.8863	0.4407	871	514	357	759	0.9950	0.5901	0.4038	0.4795
glm_encoded	top_0.5pct	0.8863	0.4407	1126	556	570	717	0.9943	0.4938	0.4368	0.4635
glm_base	best_f1	0.8375	0.3429	830	479	351	794	0.9949	0.5771	0.3763	0.4555
glm_base	top_0.5pct	0.8375	0.3429	1126	526	600	747	0.9940	0.4671	0.4132	0.4385

There is a clear threshold tradeoff. A top-0.5% rule is stricter and therefore has higher precision. A top-1% or top-2% rule catches more fraud but sends more legitimate transactions to review.

One useful detail is that the encoded random forest did *not* win. I expected the identity-style encodings — merchant, card, job, category, and state rate features — to help, because repeated merchants and cards can carry history. In this run, the plain random forest using structured features did better on validation F1, so the final choice is the simpler model, picked on validation before the test set was touched.

8 Final Held-Out Test

After selecting the model and threshold on validation, the model was refit using the full prepared training sample. The final evaluation below is on the held-out test rows.

Table 8: Final held-out test performance

model	rule	roc_auc	pr_auc	flagged	tp	fp	tn	fn	accuracy	precision	recall	f1
rf_base	recall80	0.9936	0.8234	1228	1016	212	298411	361	0.9981	0.8274	0.7378	0.78

The confusion matrix is easier to read in words: the model flagged 1228 transactions, of which 1016 were fraud and 212 were not. It missed 361 fraud cases. Since fraud is rare, this is a large concentration improvement over the base rate: the alert queue is about 180.3 times richer in fraud than the average transaction stream.

The ROC curve asks whether fraud tends to be ranked above non-fraud. The precision-recall curve is stricter for this problem: it shows how clean the alert queue remains as we try to catch more fraud.

Table 9: Selected model under alternative threshold rules on the test set

rule	flagged	alert_rate	tp	fp	fn	accuracy	precision	recall	f1
recall80	1228	0.0041	1016	212	361	0.9981	0.8274	0.7378	0.7800
top_0.5pct	1243	0.0041	1019	224	358	0.9981	0.8198	0.7400	0.7779
best_f1	1275	0.0043	1029	246	348	0.9980	0.8071	0.7473	0.7760
top_1pct	2713	0.0090	1187	1526	190	0.9943	0.4375	0.8620	0.5804
top_2pct	6076	0.0203	1288	4788	89	0.9837	0.2120	0.9354	0.3456

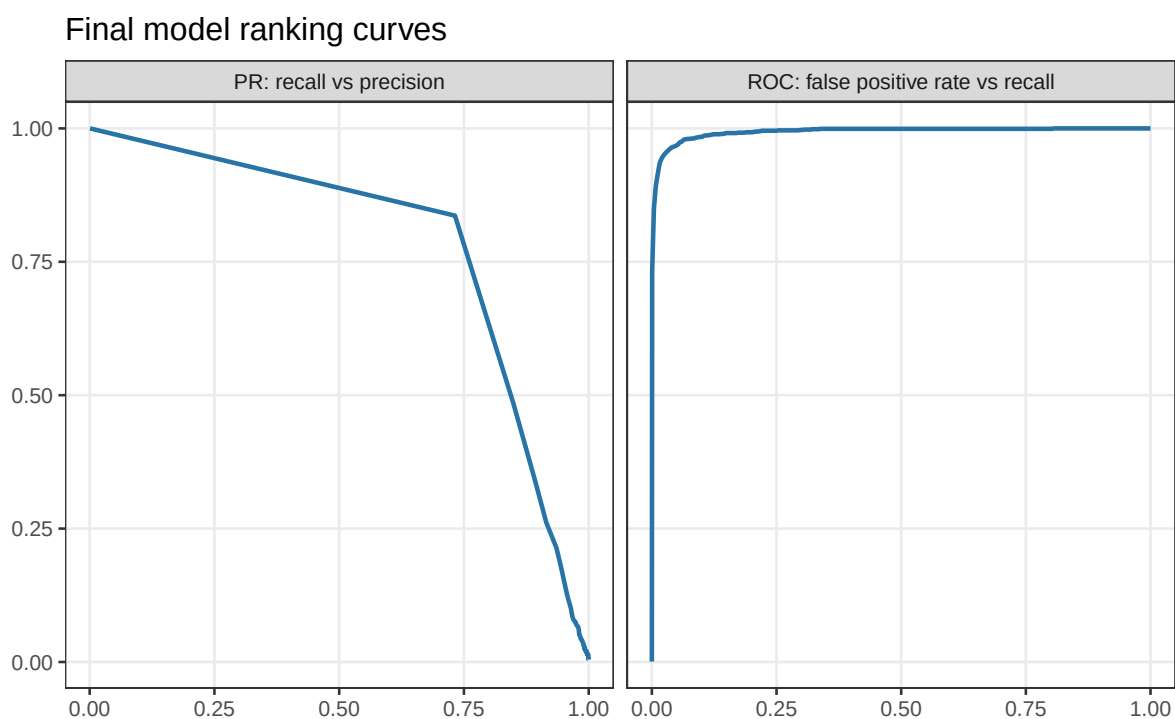


Figure 6: ROC and precision-recall views of the final model on the held-out test sample.

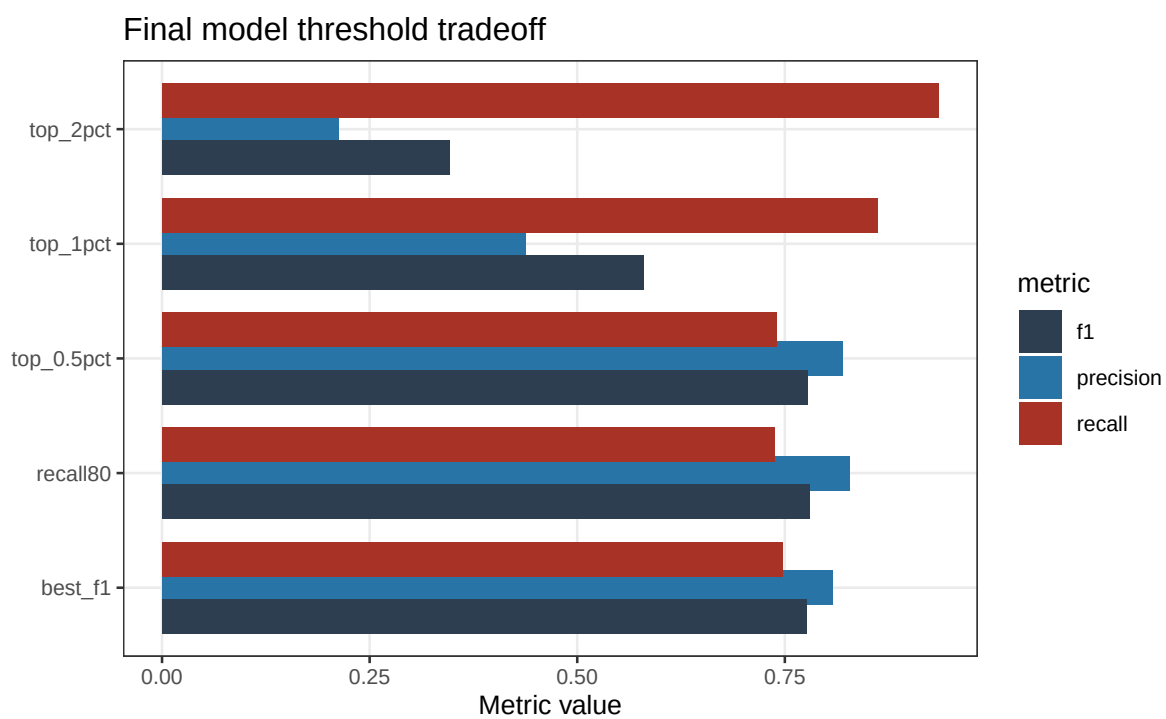


Figure 7: Precision/recall tradeoff for threshold rules on the final selected model.

The selected recall80 threshold flags 1228 test transactions in total. The top-1% rule is much wider — it flags 2713 transactions, roughly $2.2\times$ the size of the selected queue — and catches 0.862 of fraud at precision 0.438. That tradeoff could still be acceptable if a review team can handle the larger queue and the cost of missing fraud outweighs the cost of investigating extra transactions.

9 Inspecting Individual Predictions

Aggregate metrics are necessary, but they can make the model feel abstract. I also inspected individual test transactions to see what the alert queue actually looks like. The table below shows the highest-scored transactions in the test set.

Table 10: Highest-risk transactions according to the final model

	outcome	score	amt	category	hour	age_years	distance_km	gender	state
1768	true positive	1	780.52	misc_net	22	61.8	66.1	M	WI
2027	true positive	1	955.16	shopping_net	23	61.8	53.9	M	WI
11667	true positive	1	790.54	misc_net	22	61.2	81.2	M	NY
11670	true positive	1	723.23	misc_net	22	61.2	96.1	M	NY
11771	true positive	1	902.30	shopping_net	23	61.2	65.5	M	NY
11781	true positive	1	741.77	shopping_pos	23	61.2	105.7	M	NY
56270	true positive	1	333.89	grocery_pos	22	52.9	53.0	M	NE
58565	true positive	1	979.00	shopping_net	22	52.9	42.8	M	NE
58597	true positive	1	1133.22	shopping_net	22	52.9	96.2	M	NE
58708	true positive	1	1166.00	shopping_net	23	59.5	78.5	F	MS
58732	true positive	1	953.63	shopping_net	23	52.9	108.6	M	NE
58773	true positive	1	882.13	shopping_net	23	52.9	99.6	M	NE

Most of the highest-scored alerts are true positives — these are the rows that would land at the top of an analyst’s review queue.

9.1 Correct Alerts

Table 11: Examples of fraud cases correctly placed in the alert queue

	outcome	score	amt	category	hour	age_years	distance_km	gender	state
1768	true positive	1	780.52	misc_net	22	61.8	66.1	M	WI
2027	true positive	1	955.16	shopping_net	23	61.8	53.9	M	WI
11667	true positive	1	790.54	misc_net	22	61.2	81.2	M	NY
11670	true positive	1	723.23	misc_net	22	61.2	96.1	M	NY
11771	true positive	1	902.30	shopping_net	23	61.2	65.5	M	NY
11781	true positive	1	741.77	shopping_pos	23	61.2	105.7	M	NY
56270	true positive	1	333.89	grocery_pos	22	52.9	53.0	M	NE
58565	true positive	1	979.00	shopping_net	22	52.9	42.8	M	NE

These examples show the model catching fraud across specific transaction contexts rather than merely repeating the base rate. They also help explain why an alert model can be useful even when it is not perfect: a small number of reviewed transactions contains many true fraud cases.

9.2 False Alerts and Missed Fraud

No fraud model is perfect, so the errors matter. False positives are legitimate transactions the model would send to review. Missed fraud cases are fraudulent transactions that scored below the selected threshold.

Table 12: Examples of false alerts

	outcome	score	amt	category	hour	age_years	distance_km	gender	state
157708	false positive	1.000	1010.41	shopping_net	22	29.9	89.1	M	TX
58627	false positive	0.994	906.21	shopping_net	22	22.0	102.6	M	NY
81650	false positive	0.983	791.99	shopping_net	22	22.0	33.6	M	NY
224763	false positive	0.983	284.77	grocery_pos	3	27.9	82.0	F	OH
50217	false positive	0.972	774.23	misc_net	1	29.6	74.4	M	TX
238012	false positive	0.972	908.19	misc_net	2	28.1	71.8	F	TX
6547	false positive	0.967	885.07	misc_net	3	36.0	109.4	M	TX
21705	false positive	0.961	1029.49	shopping_pos	22	32.9	102.3	F	PA

Table 13: Examples of missed fraud cases

	outcome	score	amt	category	hour	age_years	distance_km	gender	state
155850	missed fraud	0.761	1319.97	shopping_pos	2	46.6	86.9	F	NE
23016	missed fraud	0.733	1071.19	shopping_net	6	34.2	26.4	F	AR
178389	missed fraud	0.639	1054.16	shopping_net	7	49.9	96.7	F	TX
47435	missed fraud	0.711	1041.39	shopping_net	9	32.8	61.1	F	NC
270826	missed fraud	0.722	1029.47	shopping_pos	11	32.1	108.5	F	WY
151528	missed fraud	0.794	1003.86	shopping_net	2	49.9	105.0	F	OK
27437	missed fraud	0.717	989.92	shopping_net	10	34.2	52.3	F	AR
283423	missed fraud	0.717	983.27	shopping_net	9	42.8	112.5	F	LA

These error examples make the tradeoff concrete. Tight thresholds produce a cleaner queue but miss more fraud. Wider thresholds catch more fraud but include more legitimate transactions. The right choice depends on review capacity and the cost of missing fraud versus interrupting a legitimate customer.

10 Discussion

The model is useful because it turns a very imbalanced stream into a concentrated review queue. In the test sample, the overall fraud rate is 0.00459; the selected alert rule has precision 0.827, so a flagged transaction is far more likely to be fraud than a random one.

The examples in §9 sharpen this picture. The top alerts are not random low-dollar transactions — they tend to combine high transaction amounts with categories and hours that the exploratory analysis already flagged. The false positives sit in the same fields, which is exactly why they are hard: the model is not

making careless mistakes; it is hitting a real ambiguity boundary. The missed-fraud cases tell the opposite story — they tend to look ordinary on the features the model has, which means recovering them would likely require signals outside this dataset (transaction velocity, account history, device or session) rather than a smarter model on the same columns.

A few methodological caveats apply to the metrics above:

- **Sample size.** This report uses a sample of the provided files (see §12) rather than the full Hugging Face train and test sets. The workflow can be re-run at full scale by raising the `CC_TRAIN_N` and `CC_TEST_N` environment variables; the exact numbers will move slightly, but the model-progression pattern (amount $\rightarrow$ linear $\rightarrow$ non-linear) is robust.
- **In-fold target encoding.** The `glm_encoded` and `rf_encoded` variants use rate encodings computed from the same fold they train on. That is acceptable for an exploratory comparison, but production target encoding should be out-of-fold so each training row receives an encoding that was not built from its own label. The selected `rf_base` model does not use these encodings, so this caveat does not affect the final test result.
- **Random split, not time-ordered.** The provided train/test split is random rather than time-ordered. Real deployment would require a time-ordered backtest to detect drift in fraud patterns over time.
- **Predictive performance only.** Cost-sensitive thresholding, fairness checks across demographics, and manual review capacity all matter for a real deployment but are out of scope for this report.

11 Conclusion

Three takeaways from this run:

1. **The product is concentration, not accuracy.** The selected alert rule is roughly $180\times$ more fraud-rich than the underlying transaction stream. That is the practical thing a bank could do with this model — narrow 300,000 transactions down to a 1228-row review queue with 0.827 precision — and it is what the metrics in §1 quantify.
2. **Non-linearity, not richer features, drove the lift.** Adding identity/entity rate encodings on top of logistic regression produced only a modest gain, but switching to a random forest produced the largest single jump in validation F1. The encoded random forest did not beat the plain one in this run; the simpler model won on its own merits before the test set was touched.
3. **Failure has a shape.** The missed-fraud and false-positive tables in §9 are not random — false positives concentrate in the same high-risk categories the model learned to flag, and missed fraud tends to look ordinary in amount and category. Recovering those cases would require signals not in this dataset (transaction velocity, account history, device or session features) rather than a different algorithm on the same features.

The methodological caveats — sampled rather than full files, in-fold target encoding for the encoded variants, and the provided random split rather than a time-ordered backtest — apply to all the metrics above and are discussed in §10. None of them affect the central concentration result.

12 Reproducibility

12.1 How to reproduce these results

Prerequisites

- R 4.4 or newer with packages `readr`, `dplyr`, `ggplot2`, `knitr`, `randomForest`, `rmarkdown`
- `pandoc` (e.g. `brew install pandoc` on macOS)
- A LaTeX engine — easiest path is `install.packages("tinytex"); tinytex::install_tinytex()`
- `curl` and `unzip` for the data download script

Steps

The simplest path is the main submission script. It downloads data if needed, prepares features, runs the model experiment, and renders the PDF:

```
CC_TRAIN_N=900000 CC_TEST_N=300000 Rscript credit_card_fraud_main.R
```

The helper scripts can also be run one by one:

1. Download the Hugging Face train/test CSVs into `data/`:

```
./download_data.sh
```

2. Build the prepared feature tables. `CC_TRAIN_N` / `CC_TEST_N` cap the rows read from each CSV (raise or unset for the full files):

```
CC_TRAIN_N=900000 CC_TEST_N=300000 Rscript prepare_credit_card_data.R
```

3. Fit the model candidates, select on validation, and score the held-out test set:

```
Rscript experiment_credit_card_models.R
```

4. Render this PDF report:

```
Rscript render_submit_pdf.R
```

What gets produced

- `cc_fraud_data.RData` — prepared feature tables (`cc_train`, `cc_test`, `meta`)
- `cc_fraud_experiments.RData` — validation, test, importance, and example tables
- `Credit_card_fraud_report.pdf` — this document

For edX submission, `credit_card_fraud_main.R` is the main R script. The helper scripts remain in the project folder to keep the code easier to review.

12.2 Determinism

The internal `cc_train` validation split uses `set.seed(1)`. Each random-forest fit uses a fixed seed (1001 for `rf_base`, 1002 for `rf_encoded`), so two runs at the same `CC_TRAIN_N` / `CC_TEST_N` produce identical metrics. Changing the sample sizes will move the numbers slightly because the underlying training rows change, not because of model randomness.

12.3 Run footprint

The numbers in this PDF were produced under R 4.5.2 against 900,000 prepared training rows and 300,000 held-out test rows. Wall-clock time for the experiment step scales roughly with training rows; expect a few minutes at the default sample size on a modern laptop, longer at full file size.

13 References

- HarvardX. *PH125.9x Data Science: Capstone — Choose Your Own Project Submission and Grading Rubric*. edX course page, accessed May 2026.
- pointe77. *credit-card-transaction*. Hugging Face Datasets. <https://huggingface.co/datasets/pointe77/credit-card-transaction>
- Dal Pozzolo, A., Caelen, O., Johnson, R. A., and Bontempi, G. (2015). *Calibrating probability with undersampling for unbalanced classification*. IEEE Symposium Series on Computational Intelligence.
- Breiman, L. (2001). *Random Forests*. Machine Learning, 45, 5-32.
- Liaw, A. and Wiener, M. (2002). *Classification and Regression by randomForest*. R News, 2(3), 18-22.
- Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C., and Bontempi, G. *Reproducible Machine Learning for Credit Card Fraud Detection: Practical Handbook*. <https://fraud-detection-handbook.github.io/fraud-detection-handbook/>